Facultad de Ingeniería
Programación Orientada a Objetos
Segundo Ciclo 2026

Nombre: Jonathan David Delgado Chavarria **Carné: 261861** **Fecha: 08/11/2026**

Análisis – Ejercicio de Polimorfismo

1. Descripción del problema

Se solicita construir un sistema centralizado que gestione las conexiones entre el "backend" de nuestro comercio electrónico y los distintos servicios externos a través de sus "APIs" (sepa Dios que es eso).

En una plataforma de comercio electrónico, tendremos que interactuar con distintos proveedores que ofreceran distintos tipos de servicios (por ejemplo, pago de pedidos, gestión de envíos, documentación legal, etc...). Los distintos proveedores ofrecen distintas funcionalidades y no siempre pueden realizar todas las operaciones.

Para solucionarlo, es necesario crear un sistema que gestione, de manera uniforme, todas las integraciones. Para lograrlo es necesario crear interfaces que describan los comportamientos que puede tener cada tipo de servicio.

De esta manera podemos añadir nuevos proveedores sin tener que modificar la lógica principal de nuestra aplicación. Además, tenemos que poder listar, buscar y ordenar las integraciones que tenemos registradas.

2. Objetivo del sistema

Como objetivo del sistema, se quiere desarrollar un administrador de integraciones externas. Para ello, se utilizara una estructura basada en herencia, interfaces y polimorfismo. De esta manera se van a poder manejar distintas APIs sin demasiadas complicaciones.

Como requisito, se debe asegurar que sólo sean implementadas las funcionalidades propias de cada integración de manera individual. De esta forma se podrá añadir nuevo proveedor sin alterar el sistema principal o generando algún tipo de dependencia innecesaria entre los elementos.

3. Tipos de integraciones

De acuerdo al planteamiento del problema podemos distinguir los siguientes tipos de servicios externos:

- Pasarelas de pago. Servicio que permite procesar cobranza y reintegros (Contrato transaccional)

- Proveedores logísticos. Servicio que permite consulta y seguimiento de envíos (Contrato consultable)

- Certificadores FEL. Servicio que permite elaborar documentos legales (Contrato documentable)

También existe un contrato que sirve para llevar registro de las operaciones que realiza la integración, denominado contrato auditable. Por otro lado, se agrega la interfaz Consultable como extensión del diseño. De esta forma se pueden modelar explícitamente aquellos servicios que son sólo de consulta. No están obligados a implementar operaciones financieras o de documento.

4. Requisitos funcionales

1. Se debe crear en un inicio un catálogo de integraciones (mínimo de 10 conexiones a APIs de distintos tipos).

2. Debe mostrarse el listado de todas las integraciones registradas (con su estado de conexión).

3. Debe ser posible buscar por el identificador de la integración, mostrando sus parámetros de configuración.

4. Las integraciones deben poder ser ordenadas por el tiempo promedio de latencia (en milisegundos).

5. Debe permitir ejecutar cobros y reintegros (mediante integraciones que implementen el contrato Transaccional).

6. Debe ser posible consultar información de los servicios, que implementan el contrato Consultable.

7. Debe ser posible generar documentos, mediante la integraciones que implementen Documentable.

8. Debe ser posible verificar las operaciones mediante el contrato Auditable.

9. Las integraciones deben soportar ignorar las peticiones de las operaciones que no soporten (de forma que no afecte al correcto funcionamiento del sistema).

10. El sistema debe ser extensible (se pueden agregar nuevas integraciones sin tener que modificar la lógica central del catálogo/sistema).

5. Datos principales del sistema

En cada integración se deberá incluir cierta información que nos permitirá identificar y gestionar dicha integración, los atributos serian:

Identificador: Un identificador único de la integración.

Nombre: El nombre del proveedor o servicio.

Tipo: Tipo de categoría de la integración.

Estado: Nos indica si esta conexión está activada o desactivada.

Latencia promedio: Es el tiempo medio de respuesta (en milisegundos).

Parámetros de configuración: Los parámetros que hagan falta para identificar y configurar la conexión.

Dependiendo del proveedor, los parámetros de configuración pueden ser distintos. Por ejemplo, en una pasarela de pago puede ser necesaria una clave de API. En un proveedor logístico puede ser necesario un código de seguimiento o bien una dirección de servicio. Son los datos comunes de todas las integraciones. Las operaciones propias de cada una están definidas a través de sus interfaces.

6. Restricciones y consideraciones

Para asegurar un correcto funcionamiento del sistema debemos tener en cuenta los siguientes aspectos:

- Todos los tipos de integraciones deben guardarse en una misma lista polimórfica.

- Las clases deben tener en común una estructura hereditaria).

- Cada tipo de integración sólo ha de implementar aquellas interfaces que sean compatibles con ella.

- Hay que ignorar aquellas operaciones que sean incompatibles. Para ello hay que realizar comprobaciones de compatibilidad (evitar excepciones en tiempo de ejecución).

- Se debe ordenar por el atributo "latencia promedio".

- La aplicación ha de cumplir con el patrón MVC (para separar datos, interfaz de usuario y lógica de control).

7. Solución propuesta

La propuesta consiste en crear una clase abstracta (IntegracionAPI) donde se definirán los atributos y comportamientos genéricos a todos los proveedores. A partir de esta se crearán tres clases concretas; PasarelaPago, Proveedor_Transporte y Certificador_FEL.

Los aspectos específicos se definirán a través de las interfaces; Transaccional, Consultable y Documentable. La clase base también implementará la interfaz Auditable para poder llevar un registro de todas las operaciones realizadas en las integraciones.

El manejo de los objetos se hará a través de una única lista (List) en la clase Catalogo_Integ. De esta forma se podrá manejar distintos tipos de proveedor y acceder a ellos de manera polimórfica.

Para separar el manejo del catálogo, el procesamiento de las solicitudes y la interacción con el administrador del sistema se utilizarán los componentes de MVC.

Declaración de Uso IA: Se utilizó ChatGPT como consejero de programación y generador de imágenes para el diagrama UML.